

Technical Report: Peer-to-Peer Distributed File Sharing System

1. Introduction

This project implements a **peer-to-peer distributed file sharing system** using a **tracker-client architecture**. It allows users to form groups, share files, and download them concurrently from multiple peers.

The system ensures:

- **File integrity** using SHA1 hashes for both files and file pieces.
- **Concurrency** in downloads from multiple peers.
- **High availability** via synchronized trackers.
- **Seamless fault-tolerance** in case of tracker failures.

2. Implementation Approach

2.1 Tracker Servers

- Two trackers are used for redundancy and high availability.
- Trackers maintain:
 - Registered users and their credentials
 - Groups, members, and pending join requests
 - File metadata and piece hashes
- Trackers communicate with clients for user/group operations and metadata retrieval.
- Synchronization between trackers is done via a dedicated TCP connection (tracker_sync_listener) to propagate all operations consistently.
- Each client connection is handled in a separate thread to support multiple simultaneous clients.

2.2 Clients

- Clients act as both **seeders** and **downloaders**.
- Each client:

- Connects to one of the trackers (auto-reconnects if tracker fails)
- Issues commands for user/group/file operations
- Downloads file pieces from multiple peers concurrently
- Verifies SHA1 hashes for integrity before saving pieces
- Clients store only necessary metadata locally for session management and ongoing downloads.

2.3 File Management

- Files are split into **512KB pieces**, except the last piece which may be smaller.
- SHA1 hashes are calculated for each piece and the complete file.
- Trackers maintain a map of files shared in each group, including piece hashes and seeder information.

3. Synchronization Algorithm

3.1 Tracker-to-Tracker Synchronization

- Any update on a tracker (user creation, group modification, file upload) is sent to the other tracker as a **sync message**.
- Messages are **ordered and atomic** to maintain consistency.
- Each tracker applies received operations in the same order as performed locally.

3.2 Failure Handling

- If a tracker is temporarily offline:
 - Sync messages are queued and resent when it reconnects.
 - Clients continue to operate with the available tracker.

4. Piece Selection Strategy

- **Sequential Piece Downloading:** Pieces are requested in order from multiple peers.
- **Concurrent Downloads:** Each piece may be downloaded from a different peer thread to maximize bandwidth usage.
- **Corrupted Piece Handling:** If a downloaded piece fails SHA1 verification, it is discarded and re-requested from another peer.
- **Rationale:** Sequential downloading ensures simpler implementation while still allowing multiple concurrent threads for performance.

5. Protocol Design

5.1 Client ↔ Tracker Protocol

- TCP-based communication
- Text-based commands terminated by \n

5.2 Tracker ↔ Tracker Protocol

- TCP sync connection
- Ordered messages propagate updates
- Handles partial transmissions gracefully using buffer management
- Ensures consistency across trackers

5.3 Client ↔ Client Protocol

- TCP-based piece transfer
- Client requests a piece by number
- Seeder sends piece data; downloader verifies SHA1

6. Challenges Encountered and Solutions

Challenge	Solution
Concurrency in downloads	Implemented multithreading with mutex locks to manage shared data
Tracker failures	Added auto-reconnect mechanism and redundant tracker synchronization
File integrity	SHA1 verification per piece; corrupted pieces re-downloaded
Large file handling	Streamed piece-by-piece instead of loading entire file in memory
Synchronization consistency	Ordered sync messages with queueing to ensure same state across trackers

7. Data Structures

- **User Map:** map<string, string> – maps user ID to password
- **Groups Map:** map<string, Group> – stores group information, members, owner, pending requests
- **Files Map:** map<string, vector<FileInfo>> – stores file metadata per group

Structs:

```
struct Group {  
    string group_id;  
    string owner;  
    vector<string> members;  
    vector<string> pending_requests;  
};
```

```
struct FileInfo {  
    string filename;  
    string owner;  
    string group_id;  
    vector<string> piece_hashes;  
    string full_file_hash;  
};
```

8. Implementation Details

- **Languages:** C++17
- **Libraries:** POSIX sockets, pthreads, OpenSSL EVP for SHA1
- **Directories:**
 - **tracker/:** tracker_server.cpp, operations.cpp, file_operations.cpp, headers, Makefile
 - **client/:** main.cpp, tracker_connector.cpp, file_utils.cpp, headers, Makefile
- **Compilation:** make in respective directories
- **Execution:**
 - Tracker: ./tracker tracker_info.txt tracker_no
 - Client: ./client <IP>:<PORT> tracker_info.txt

9. Testing Procedures

- **Functional Testing:** All commands (user/group/file) tested individually
- **Concurrent Clients:** 2–3 clients tested simultaneously
- **Tracker Failures:** Verified system continues with one tracker offline
- **File Sizes:** Tested files <512KB, ~10MB, ~1GB

- **Download Integrity:** Verified SHA1 of all downloaded files
- **Synchronization:** All operations on one tracker reflected in the other tracker correctly

10. Assumptions and Limitations

- Only **two trackers** supported
- No **persistent storage**; all data lost on restart
- **Network messages unencrypted**
- Sequential piece selection used
- Clients and trackers assumed to be running on reliable POSIX environment